

LFCX:py2fr: ein kleiner Python Refresher für die Ausbildung zur Fachinformatikerin

Karsten Reincke

Gewerbliche Schulen Dillenburg

11. August 2026

*Diese Präsentation stammt aus dem OER-Projekt **proTirone**. Es wurde auf Basis von **proScientia.ltz** entwickelt und ist **CC-BY-4.0** lizenziert. Gemäß CC-BY-4.0/§5 werden proTirone-Materialien so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung.*



::py2rf:Funktion aus LF09/09 als Ausgangspunkt

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2
3  import ipaddress
4
5  # insert a(n) (in)valid net description
6  IP_ADDR='192.168.0.2'
7  SN_MASK='255.255.255.0'
8  NT_ADDR='192.168.2.0'
9
10 # Rule: IP_ADDR & SN_MASK -> NT_ADDR:
11
12 ip_addr_int=(int)(ipaddress.ip_address(IP_ADDR))
13 nt_addr_int=(int)(ipaddress.ip_address(NT_ADDR))
14 sn_mask_int=(int)(ipaddress.ip_address(SN_MASK))
15
16 if ((ip_addr_int&sn_mask_int)==nt_addr_int):
17     print("net data fulfills IP_ADDR & SN_MASK -> NT_ADDR!")
18 else:
19     print("net data does not fulfill IP_ADDR & SN_MASK -> NT_ADDR!")
20
```



```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2
3  import pandas as pd
4
5  message = "Hello World"
6  print(message)
```

::py2rf:dann einige Tests auf Netzzugehörigkeit:





::py2rf: eine in-the-same-network-Funktion

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2  import ipaddress
3  import sys
4  # set the default values and overload them by cmdline arguments
5  IP_AD1='10.75.65.222'
6  NT_MSK='255.255.224.0'
7  IP_AD2='10.75.70.108'
8  if ((len(sys.argv)>1)) : IP_AD1=sys.argv[1]
9  if ((len(sys.argv)>2)) : NT_MSK=sys.argv[2]
10 if ((len(sys.argv)>3)) : IP_AD2=sys.argv[3]
11
12 # decider: check by comparing 2 results of 'netmask & ipaddress'
13 def in_the_same_network(ip_addr1,net_mask,ip_addr2):
14     return( ( (int)(ipaddress.ip_address(ip_addr1))
15               & (int)(ipaddress.ip_address(net_mask)))
16             == ( (int)(ipaddress.ip_address(ip_addr2))
17                  & (int)(ipaddress.ip_address(net_mask))) )
18
19 # main
20 print("Is ", IP_AD1, " in the same network as ", IP_AD2, "? ",
      (in_the_same_network(IP_AD1,NT_MSK,IP_AD2)))
```



::py2rf: eine bedingte in-the-same-network-Funktion

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2  import ipaddress, sys
3  # set default values and and overload them by cmdline arguments
4  IP_AD1,NT_MSK,IP_AD2 = '192.168.0.2','255.255.255.0','192.168.0.14'
5  if ((len(sys.argv)>1)) : IP_AD1=sys.argv[1]
6  if ((len(sys.argv)>2)) : NT_MSK=sys.argv[2]
7  if ((len(sys.argv)>3)) : IP_AD2=sys.argv[3]
8
9  # decider: check by comparing 2 results of 'netmask & ipaddress'
10 def in_the_same_network(ip_addr1,net_mask,ip_addr2):
11     return( ( (int)(ipaddress.ip_address(ip_addr1))
12               & (int)(ipaddress.ip_address(net_mask)))
13             == ( (int)(ipaddress.ip_address(ip_addr2))
14                  & (int)(ipaddress.ip_address(net_mask))) )
15
16 # main
17 if ((not(IP_AD1 == IP_AD2)) and (IP_AD1 != IP_AD2)):
18     print("Is ", IP_AD1, " in the same network as ", IP_AD2, "? ",
19           (in_the_same_network(IP_AD1,NT_MSK,IP_AD2)))
20 else:
21     print("{IP_AD1} and {IP_AD2} are the same. Test is unnecessary.")
```



::py2rf: eine is-member-Methode in einer Klasse (1)

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2  import ipaddress, numpy
3
4  class ProTiCo_Network_Data:
5      def __init__(self, ia_str='192.168.0.2', nm_str='255.255.255.0'):
6          self._ia_str, self._nm_str=ia_str, nm_str
7          self._ip_addr=int(ipaddress.ip_address(ia_str))
8          self._net_mask=int(ipaddress.ip_address(nm_str))
9          self._net_addr=(self._ip_addr&self._net_mask)
10         # BCAddr = all bits of net-address o all bits of inverse net-mask
11         self._bc_addr=( numpy.array(~(self._net_mask))
12                          .astype(numpy.int32).item() | (self._net_addr))
13         self._gw_addr=(self._net_addr+1)
14     def get_ia_str(self): return self._ia_str
15     def get_nm_str(self): return self._nm_str
16
17 if __name__ == "__main__":
18     mnw=ProTiCo_Network_Data()
19     print(f"given: {mnw.get_ia_str()}/{mnw.get_nm_str()}")
20     print(f"→ na:{mnw._net_addr}, bc:{mnw._bc_addr}, gw:{mnw._gw_addr}")
```



::py2rf: eine is-member-Methode in einer Klasse (2)

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2
3  import ipaddress
4  import mod_py2fr_nwcore as nwc
5
6  class ProTiCo_Network(nwc.ProTiCo_Network_Data):
7
8      def __init__(self,ia_str='192.168.0.13',nm_str='255.255.255.128'):
9          super().__init__(ia_str,nm_str)
10
11      def _show_address(self,type,address):
12          addr_str=ipaddress.ip_address(address)
13          print(f"{type} hat den Wert <{addr_str}> = <{address:b}>")
14
15      def describe(self):
16          self._show_address("ipv4-addr",self._ip_addr)
17          self._show_address("net-mask",self._net_mask)
18          self._show_address("net-addr",self._net_addr)
19          self._show_address("bc-addr",self._bc_addr)
20          self._show_address("gw-addr",self._gw_addr)
21
22      def is_member(self,ipv4_str):
23          test_ip=(int)(ipaddress.ip_address(ipv4_str))
24          return ((test_ip&self._net_mask)==self._net_addr)
25
26  if __name__ == "__main__":
27      print("starting the module self test:")
28      nw=ProTiCo_Network()
29      print(f"nw-data: {nw.get_ia_str()}/{nw.get_nm_str()}")
30      nw.describe()
31      print(f"Is 127.0.0.1 in mnw? {nw.is_member('127.0.0.1')}")
```




::py2rf: eine is-member-Methode in einer Klasse (3)

```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2
3  import sys
4  import mod_py2fr_nw as dnw
5
6  # main:
7
8  start_ia_str="192.168.2.42"
9  start_nm_str="255.255.255.0"
10
11  if ((len(sys.argv)>1)) : start_ia_str=sys.argv[1]
12  if ((len(sys.argv)>2)) : start_nm_str=sys.argv[2]
13
14  nw=dnw.ProTiCo_Network(start_ia_str,start_nm_str)
15  nw.describe()
16
17  tia="127.0.0.13"
18  if ((len(sys.argv)>3)) : tia=sys.argv[3]
19
20  print(f"{tia} + {nw.get_ia_str()} in same net?: <{nw.is_member(tia)}>")
21
```

Herzlichen Dank für Ihre Aufmerksamkeit!



Open Educational Resources

*Dieses Dokument stammt aus dem OER-Projekt <https://github.com/protirone/>, initiiert v. **Karsten Reincke, Hohenahr**. Es stellt **freie Unterrichtsmaterialien** samt **Quellcode** unter den Bedingungen der **CC-BY-4.0-Lizenz** zur Verfügung: Sie werden so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

Bildnachweise

-  von Wikimedia Commons. **Lizenziert** unter **CC0**. Bereitgestellt auf **Wikimedia**.
-  von Karsten Reincke. **Lizenziert** unter **proTirone-Logo-License**. Bereitgestellt auf **github**. (may only be used as logo for proTirone)
-  von anonymous. **Lizenziert** unter **CC0**. Bereitgestellt auf **Pxhere:519043**.